

WaTrck

WhatsApp Number Change and Webhook Migration Guide

Safe process for changing the production WhatsApp number without bypassing Firebase or breaking conversion tracking

Verified against WaTrck branch: New-payload-fixes
Firebase project: aida-muscat-wa-tracking
Updated: September 2026

Critical architecture rule: Meta must deliver inbound WhatsApp webhooks to Firebase first. Firebase performs token matching, conversion updates, Firestore backup, and then forwards the same raw payload to Chatwoot. Never point Meta directly to Chatwoot for the production flow.

1. What this guide protects

Changing the WhatsApp number is not only a landing-page edit. The number participates in the customer click path, the Meta Cloud API configuration, the Firebase webhook bridge, and Chatwoot. The previous outage happened because a phone-number-level Meta webhook override sent traffic directly to Chatwoot, bypassing Firebase. Chatwoot still received messages, but WaTrck could not consume the tracking token and complete conversions.

Correct production flow



Do not accept this flow: Meta -> Chatwoot directly. It can look healthy in Chatwoot while silently breaking WaTrck conversion attribution.

Current reference points in WaTrck

Item	Current reference
Local repo	D:\Real Estate Projects\Advanced Tracking System\NewWaTrck
Firebase project	aida-muscat-wa-tracking
Firebase inbound webhook	https://us-central1-aida-muscat-wa-tracking.cloudfunctions.net/whatsappWebhook
Current Chatwoot bridge URL in functions/index.js	https://wa-chat.seaborn-properties.com/webhooks/whatsapp/+201034285454
Current number used by landing pages	+201034285454

Treat the current number above as a reference only. During a number change, replace it with the new E.164 number and use the new Meta Phone Number ID. Do not reuse the old Phone Number ID.

2. Information to collect before touching production

- New WhatsApp phone number in E.164 format, including the leading +, for example +201XXXXXXXXX.
- Meta WhatsApp Business Account ID (WABA_ID) that owns the new number.
- Meta Phone Number ID (PHONE_NUMBER_ID) for the new number. This is not the visible phone number.
- Meta access token with permission to manage the WABA and phone number. Do not save the token in GitHub or in this guide.
- The existing Firebase verification token value, or a planned replacement. WaTrck reads it from Secret Manager as whatsapp_verify_token.
- The Chatwoot inbound webhook URL for the new number/inbox. Do not guess it if Chatwoot gives you a specific endpoint.
- Access to Webflow/landing pages so every CTA can be updated and republished.
- Firebase CLI access on the deployment machine.

Recommended change window: Prepare Chatwoot and Meta first, then deploy Firebase, then update/publish landing pages. This reduces the time in which users can click an old destination.

Step 1 - Open a fresh CMD and inspect every old-number reference

Start from a normal Windows CMD prompt. The /d switch is important because the repo is on a different drive.

```
cd /d "D:\Real Estate Projects\Advanced Tracking System\New\WaTrck"
dir firebase.json
git status
git branch --show-current
findstr /S /N /I "201034285454" *.*
```

The search is your safety net. At minimum, expect the old number in the Firebase Chatwoot bridge URL and in the landing-page WhatsApp configuration. Depending on the current repo state, there may be more occurrences.

Before editing: Make sure you are on the intended branch and there are no unrelated uncommitted changes. Back up or commit the current working state first.

Step 2 - Prepare Chatwoot for the new WhatsApp number

The Firebase function forwards Meta's raw webhook payload to a Chatwoot webhook URL. The current code hardcodes the old number inside that URL. Before changing Firebase, make sure Chatwoot has a working inbox/integration for the new number and copy its inbound webhook URL exactly.

- Create or update the WhatsApp inbox/integration in Chatwoot for the new number.
- Confirm the new number is connected to the correct Meta WABA and app.
- Copy the actual Chatwoot inbound webhook URL for that number.
- Do not point Meta directly to this Chatwoot URL. Firebase remains the Meta callback.

Current code reference: functions/index.js currently uses `CHATWOOT_WEBHOOK_URL = "https://wa-chat.seaborn-properties.com/webhooks/whatsapp/+201034285454"`. Replace it with the endpoint for the new number.

Step 3 - Update the Firebase Chatwoot bridge and deploy whatsappWebhook only

In functions/index.js, change only the Chatwoot bridge URL unless your new Chatwoot setup requires another deliberate change. Keep the token logic, BSUID/username handling, Firestore paths, and forwarding mechanism intact.

```
const CHATWOOT_WEBHOOK_URL =
  "https://wa-chat.seaborn-properties.com/webhooks/whatsapp/+NEW_NUMBER";
```

Then deploy only the webhook function:

```
cd /d "D:\Real Estate Projects\Advanced Tracking System\New\WaTrck"
firebase use aida-muscat-wa-tracking
set FUNCTIONS_DISCOVERY_TIMEOUT=60
firebase deploy --only functions:whatsappWebhook
```

If Firebase CLI asks you to authenticate:

```
firebase login
```

If backend discovery still times out, retry in the same CMD with:

```
set FUNCTIONS_DISCOVERY_TIMEOUT=120
firebase deploy --only functions:whatsappWebhook
```

Why only whatsappWebhook?: This change should not redeploy unrelated conversion, cleanup, or export functions. A surgical deploy reduces risk.

Step 4 - Confirm the Meta app-level callback points to Firebase

In Meta for Developers, open the app used by the WhatsApp Business Account. In the WhatsApp webhook configuration, the callback should be the Firebase function, not Chatwoot:

```
https://us-central1-aida-muscat-wa-tracking.cloudfunctions.net/whatsappWebhook
```

Use the same verification token stored in Google Secret Manager under whatsapp_verify_token. Subscribe the app/WABA to the messages webhook field.

Verification behavior: WaTrck answers Meta's GET verification challenge only when hub.verify_token matches the Secret Manager value. A mismatch produces HTTP 403 verification failed.

Step 5 - Open a fresh CMD for Meta Graph API checks

Use placeholders first. Never paste the access token into source code or commit it.

```
set META_TOKEN=YOUR_META_ACCESS_TOKEN
set GRAPH_VERSION=v24.0
set WABA_ID=YOUR_WABA_ID
set PHONE_NUMBER_ID=YOUR_NEW_PHONE_NUMBER_ID
```

If Meta has moved your app to a newer Graph API version, replace v24.0 with the version currently supported by your app. The request shapes below are the important part.

Step 6 - Ensure the app is subscribed to the WABA

```
curl -i -X POST "https://graph.facebook.com/%GRAPH_VERSION%/%WABA_ID%/subscribed_apps" ^
-H "Authorization: Bearer %META_TOKEN%"
```

A successful subscription should return an HTTP 200 response. This matters because Meta requires the app to be subscribed before alternate callback behavior can be managed reliably.

Step 7 - Inspect WABA-level and phone-number-level webhook overrides

This is the step that prevents the old failure from returning. Meta can route webhooks at more than one level. A lower-level override can silently take precedence over the callback configured in the app dashboard.

Check WABA subscriptions and any WABA-level override:

```
curl -s -X GET "https://graph.facebook.com/%GRAPH_VERSION%/%WABA_ID%/subscribed_apps" ^
-H "Authorization: Bearer %META_TOKEN%"
```

If the response shows override_callback_uri pointing somewhere other than Firebase, remove the alternate WABA callback by subscribing normally without an override body:

```
curl -i -X POST "https://graph.facebook.com/%GRAPH_VERSION%/%WABA_ID%/subscribed_apps" ^
-H "Authorization: Bearer %META_TOKEN%"
```

Check the new phone number's webhook configuration:

```
curl -s -X GET "https://graph.facebook.com/%GRAPH_VERSION%/%PHONE_NUMBER_ID%?fields=webhook_configuration" ^  
-H "Authorization: Bearer %META_TOKEN%"
```

If a phone-number-level callback points directly to Chatwoot or another endpoint, clear it. This is the exact class of override that caused the previous bypass:

```
curl -i -X POST "https://graph.facebook.com/%GRAPH_VERSION%/%PHONE_NUMBER_ID%" ^  
-H "Authorization: Bearer %META_TOKEN%" ^  
-H "Content-Type: application/json" ^  
--data '{"webhook_configuration":{"override_callback_uri":""}}'
```

Then run the GET check again:

```
curl -s -X GET "https://graph.facebook.com/%GRAPH_VERSION%/%PHONE_NUMBER_ID%?fields=webhook_configuration" ^  
-H "Authorization: Bearer %META_TOKEN%"
```

PASS condition: There must be no phone-level or WABA-level override that sends production messages directly to Chatwoot. The effective inbound route must fall back to the app-level Firebase callback.

Step 8 - Update every landing page to open the new number

Each WaTrck landing page has a visible data-wa-phone attribute and a DEFAULT_PHONE fallback. Update both. The click handler strips formatting before building the WhatsApp URL, so the config can keep normal E.164 formatting with a leading +.

```
data-wa-phone="+NEW_NUMBER"  
  
var DEFAULT_PHONE =  
  "+NEW_NUMBER";
```

Use the repo-wide search again after editing:

```
cd /d "D:\Real Estate Projects\Advanced Tracking System\New\WaTrck"  
findstr /S /N /I "201034285454" *.*
```

If the old number is intentionally retained in documentation or a backup, that is fine. There should be no active production landing or webhook configuration still using it.

- Publish the updated Webflow pages.
- Test the staging domain first if possible.
- Check both the floating WhatsApp widget and any other custom_action_cta buttons.

Step 9 - Run the end-to-end production-path test

Do not stop at "message arrived in Chatwoot." A valid migration proves that Firebase also received and processed the message.

1. Open a WaTrck landing page and click the WhatsApp CTA.
2. Confirm WhatsApp opens the NEW number.
3. Confirm the prefilled message contains Ref: #TOKEN.
4. Send the message from a real WhatsApp account.
5. Confirm the conversation appears in Chatwoot.
6. In Firestore, confirm projects/<projectId>/clicks/<TOKEN> now has used = true and used_at.
7. Confirm whatsapp_msg_id and sender identity fields were written to the click document.

8. Confirm the message was backed up under `projects/<projectId>/whatsapp_conversations/<senderId>/messages/<msgId>`.
9. Confirm `whatsappSenderId/<senderId>` points to the correct project when applicable.
10. Run the normal conversion exporter later and confirm the conversion enters the proper sheet without webhook errors.

Best proof: Chatwoot + Firestore conversion state must both succeed. Chatwoot alone does not prove WaTrck is in the path.

Optional: check Firebase logs during the test

Use Firebase/GCP logs to look for Token logic executed, Chatwoot Forward Error, verification errors, or webhook fatal errors. The webhook intentionally forwards to Chatwoot in a fire-and-forget call, so a Chatwoot forwarding failure is logged separately from the main Meta acknowledgement.

3. Fast troubleshooting matrix

Symptom	Most likely cause	What to check
Chatwoot receives messages, but Firestore click stays used=false	Meta is bypassing Firebase	Check WABA and phone webhook overrides. Clear direct Chatwoot overrides.
Meta webhook verification returns 403	Verify token mismatch	Compare Meta verify token with Secret Manager <code>whatsapp_verify_token</code> .
Firebase receives messages but Chatwoot does not	Wrong Chatwoot bridge URL or inbox	Check <code>CHATWOOT_WEBHOOK_URL</code> and logs for Chatwoot Forward Error.
Landing page still opens old number	Old number remains in page code or unpublished Webflow changes	Search <code>data-wa-phone</code> and <code>DEFAULT_PHONE</code> ; republish.
No token conversion but conversation is stored	Message did not contain a valid Ref token	Verify the first message includes Ref: <code>#TOKEN</code> and <code>tokenIndex</code> contains it.
Firebase deploy times out during function discovery	CLI discovery timeout	Set the discovery timeout to 60 seconds; retry with 120 if needed.
New number works in Meta but not Chatwoot	Chatwoot inbox/integration not configured for the new number	Verify the new Chatwoot inbox and its inbound webhook endpoint.

4. Rollback plan

If the new number fails and production traffic is affected, roll back in this order:

1. Restore the previous `CHATWOOT_WEBHOOK_URL` in `functions/index.js` and redeploy only `whatsappWebhook`.
2. Restore the previous landing-page number and republish.
3. Re-check Meta WABA and phone-level overrides so Meta still routes to Firebase, not directly to Chatwoot.
4. Send one real test message and verify both Firestore and Chatwoot before declaring rollback complete.

5. Production completion checklist

- ☐ New number is active in Meta and has its own `PHONE_NUMBER_ID`.
- ☐ Chatwoot inbox/integration for the new number is ready.
- ☐ `functions/index.js` points `CHATWOOT_WEBHOOK_URL` to the new Chatwoot endpoint.
- ☐ `functions:whatsappWebhook` deployed successfully.
- ☐ Meta app callback is the Firebase `whatsappWebhook` URL.
- ☐ WABA is subscribed to the `app/messages` webhook.
- ☐ No WABA-level override bypasses Firebase.

- [] No phone-number-level override bypasses Firebase.
- [] Every production landing page points to the new number.
- [] Old number search completed across the repo.
- [] Real test message reached Chatwoot.
- [] Same test message caused Firestore click used=true.
- [] Conversation/message backup appears under the correct project.
- [] Conversion export still works after the migration.

Appendix A - Why the Firebase webhook must stay in the middle

The current WaTrck whatsappWebhook does more than relay messages. It verifies Meta, forwards the raw payload to Chatwoot, handles hidden-phone/BSUID identities, resolves the project from the tracking token or sender index, stores the conversation, and marks the click as used. Removing Firebase from the inbound path removes all of those WaTrck functions.

```
Meta POST
-> Firebase whatsappWebhook
-> axios.post(CHATWOOT_WEBHOOK_URL, raw payload)
-> parse contacts/messages
-> extract Ref token
-> tokenIndex lookup
-> project conversation backup
-> click used=true
-> conversion_value_source='whatsapp_message'
-> HTTP 200 EVENT_RECEIVED
```

Appendix B - Meta webhook override references

The command pattern in this guide was cross-checked against Meta's WhatsApp Business Platform Postman documentation for WABA callback overrides, and against the phone-number webhook_configuration behavior used in the successful WaTrck repair.

[Meta WhatsApp Business Platform - Override Callback URL \(Postman collection\)](#)

[Meta WhatsApp Business Platform - Embedded Signup / Webhook subscriptions](#)

Note: Meta Graph API versions age over time. If v24.0 is no longer accepted, change only GRAPH_VERSION to the version supported by your Meta app; keep the same endpoint structure and validation sequence.